

What Changed in 2026 — Python Interview Intel Report

Refresh window: H2 2025 → H1 2026. Last compiled: May 2026.

This file is the field intel layer of the pack. It is not a tutorial. It is a short, dense readout of what we have been hearing from candidates and hiring managers about Python rounds over the last twelve months — what is being asked more, what is being asked less, which language features have crossed the line from trivia to expected fluency, and which interview tropes the Indian market has quietly retired.

If you came into this pack worried that Python prep gets stale fast because the language ships a new minor version every October, this section is the answer to that worry. We refresh it every six months. Everything below is keyed to Python rounds happening between June 2025 and May 2026 in the Indian market.

A small framing note before we start. Python in 2026 is in an unusual place. The 3.12 release (October 2023) shipped genuinely interview-relevant ergonomic improvements that have now had eighteen months to propagate into Indian production codebases. 3.13 (October 2024) shipped two experimental features — the free-threaded build and the JIT — that are still too new to be hiring-bar material but are showing up as "have you heard of this" probes. The async story has firmed up around `TaskGroup` and `asyncio.timeout`. Pydantic v2 finished its migration arc and is now the default assumption in Backend interviews. pandas 2.x has displaced 1.x in Data interviews and the Arrow backend has moved from curiosity to expected awareness. That is the landscape this report sits inside.

Executive Summary — the six shifts that matter most

If you only read one page of this file, read this one. These are the six shifts that have most materially changed the shape of a 2026 Python interview versus a 2024 one.

- **Decorator and generator questions are no longer "advanced" — they are baseline screening material.** A fresher in 2024 could often clear Round 1 at a Service MNC by reciting list-comprehension syntax and mutability. In 2026, the same round will probe `functools.wraps`, what runs first in a stack of decorators, and what `yield from` does. The mid-rounds have absorbed everything the senior rounds used to test, and the senior rounds have moved up into structured concurrency and typing.
- **Pydantic v2 is now the default expectation in Backend rounds.** The migration from v1 to v2 finished about eighteen months ago. Interviewers at Razorpay, PhonePe, Postman, Chargebee, and

at every GCC that runs a FastAPI service stack now assume the candidate has touched v2. The v1-to-v2 migration question itself — "what changed, what broke, why did they do it" — is being asked as a proxy for whether the candidate has shipped a real Python service in the last twelve months.

- **`asyncio.TaskGroup` has displaced `asyncio.gather` as the right-answer concurrency primitive.** PEP 654 (exception groups) provided the `ExceptionGroup` machinery that `TaskGroup` uses for error propagation; `TaskGroup` itself shipped in Python 3.11 as part of `asyncio` (no dedicated PEP), and interviewers have caught up. If a candidate writes `gather` for a structured set of awaitables in 2026, the follow-up is almost always "do you know `TaskGroup`?" The answer "yes, and here is why it is better" separates a 2026-current candidate from a 2022-trained one.
- **pandas 2.x and the Arrow backend have crossed from optional knowledge to standard data-track expectation.** `dtype_backend="pyarrow"`, copy-on-write semantics, and the `pd.NA` vs `np.nan` distinction are all interview-table-stakes at Data-role rounds. A candidate who only knows pandas 1.x patterns will be flagged as out of date within ten minutes.
- **Typing fluency separates seniors from mid-levels in Backend rounds.** `Self`, `ParamSpec`, `@override`, the new `type` statement syntax, and the disappearance of `from __future__ import annotations` as a magic incantation — these are not "nice to have" any more. They are how seniors signal seniority in 2026 rounds at Indian product cos and GCCs. The interviewer will not ask "do you know what `ParamSpec` is" — they will ask you to type a decorator correctly and watch what you reach for.
- **The GIL question has changed shape, not gone away.** Two years ago the GIL question was definitional: "what is the GIL, why does it exist, what does it block." In 2026 the question is comparative and forward-looking: "given Python 3.13's experimental free-threaded build, how does that change your reasoning about threading vs multiprocessing vs asyncio?" Candidates who recite the old definition and stop are being marked down. Candidates who recite the old definition and then mention PEP 703 and the free-threaded build with appropriate hedging are being marked up.

A seventh shift worth flagging but slightly behind the others in volume: **structural pattern matching** (`match` / `case`) is appearing more in mid-level rounds, particularly when the interviewer wants to test whether the candidate has written Python after 3.10. The volume is still smaller than the six above, but the trajectory is steep.

A few framing notes before the body of the report:

- These shifts are biased toward the Indian market between June 2025 and May 2026. US-market data, where we have visibility, mostly agrees on the direction but the magnitudes differ. Free-threaded build awareness, for example, is roughly six months ahead in the US — Indian interviewers are still treating it as bleeding edge.
- "Service MNC" in our taxonomy means TCS / Infosys / Wipro / Accenture / Cognizant / HCLTech / Tech Mahindra for the purposes of this report. "Indian product co" means Indian-headquartered consumer or B2B product companies — Razorpay, Swiggy, Zerodha, Flipkart, Paytm, PhonePe, Freshworks, Zoho, Postman, Meesho, Chargebee, BrowserStack. "GCC" means India-based

captive centres of US-headquartered firms — Microsoft India, Goldman Sachs India, JPMC India, Walmart Labs, Target India, Lowe's India, Wayfair India, AmEx India.

- We weight signal by recency. A pattern we last heard about in November 2025 is given less weight than one we have heard about in March or April 2026.
- The single biggest meta-shift behind all of these specific shifts is that Python production stacks at Indian companies have moved off 3.8/3.9 onto 3.11/3.12, and interviewer question banks have followed. That migration started roughly eighteen months ago and is still in motion — many Service MNC client projects are still on 3.9, which is why we still see 3.9-flavour questions at Service MNCs even as product cos have moved on.

Section 1 — Python 3.12 features that show up in interviews

Python 3.12 shipped in October 2023, and by mid-2026 it has had eighteen months to propagate into Indian production codebases. Indian product cos like Razorpay, Flipkart, Swiggy, and Zerodha adopted 3.12 within six months of release; Service MNC stacks lag by one to two versions, so many TCS / Infosys client projects are still on 3.10 or 3.11; GCCs sit in between and typically run 3.11 or 3.12 in greenfield work. That adoption gradient maps directly onto interview question gradients.

There are three 3.12 features that are now appearing reliably in interview rounds. Each gets its own treatment below.

1.1 PEP 695 — the new type-parameter syntax and the `type` statement

This is the single most-asked Python 3.12 feature in 2026 Backend and senior Data interviews. It is also the one candidates most often get half-right.

PEP 695 introduced a compact syntax for declaring type parameters directly on functions, classes, and aliases. Before 3.12, you wrote a generic function like this:

```
from typing import TypeVar

T = TypeVar("T")

def first(items: list[T]) -> T:
    return items[0]
```

In 3.12, the same generic is written without the standalone `TypeVar` declaration:

```
def first[T](items: list[T]) -> T:
    return items[0]
```

The bracket-after-name syntax is borrowed from C# and TypeScript and is meaningfully cleaner. The same shape extends to classes:

```
class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()
```

PEP 695 also introduced the `type` statement for type aliases. This is the part candidates most often get wrong because the old syntax — assigning to an uppercase name — still works and looks similar:

```
# Old syntax — still works but is no longer the "tell me you're current" answer
Vector = list[float]

# 3.12+ — the type statement creates a lazy TypeAliasType
type Vector = list[float]
```

The two forms are not exactly equivalent. The `type` statement creates a `TypeAliasType` object that lazily evaluates its right-hand side, which means forward references work without quoting. That detail comes up at the senior end.

How interviewers ask this: The most common framing in 2026 Backend rounds is "rewrite this 3.10 generic function in 3.12 syntax" with a small TypeVar-heavy snippet on the whiteboard. The interviewer is checking two things. First, can you write the `[T]` form fluently without looking it up. Second, do you know that the `type` statement is the new way to write aliases. A candidate who writes the function generics correctly but does the alias as `Vector = list[float]` is signalling "I have read the changelog but I have not written this in production."

What saying it correctly tells them: You have written Python after October 2023. You are following the language. You are likely a senior Python engineer, not a mid-level who just learned 3.10 and stopped.

A short example combining both:

```
type JSONValue = str | int | float | bool | None | list["JSONValue"] | dict[str, "JSONValue"]

def parse[T](raw: str, into: type[T]) -> T:
    ...
```

The forward reference inside `JSONValue` is exactly the kind of thing that requires the lazy `type` statement to work without explicit quoting in some cases. Mentioning this earns you the senior tag on the rubric.

1.2 Improved f-string semantics (PEP 701)

3.12's f-string improvements are smaller in scope than PEP 695 but they are catching candidates off guard because they invert a rule everyone had memorized. Before 3.12, you could not reuse the same kind of quote inside an f-string. You also could not put backslashes in expressions, could not put comments in expressions, and could not span multiple lines in an expression. All four of those restrictions disappeared in 3.12.

The old rule everyone learned: "outside quote and inside quote must differ." Gone.

```
# Pre-3.12: SyntaxError because the inner double-quote conflicts with the outer
# message = f"User {user["name"]} signed in"

# 3.12+: this is valid
message = f"User {user["name"]} signed in"
```

The multi-line case is the second most common interview probe:

```
# 3.12+ – multi-line expression inside f-string with a comment
report = f"""
Total: {
    sum( # accumulate all order amounts
        order.amount
        for order in orders
        if order.status == "paid"
    )
}
"""
```

How interviewers ask this: Almost never directly. The probe is usually embedded in a Guess-the-Output trap: "what does this print?" with a snippet that uses 3.12 f-string semantics. The interviewer is checking whether the candidate immediately says "this is a SyntaxError" — which it was pre-3.12 — or correctly identifies the 3.12 behaviour. A candidate who says "SyntaxError" without hedging is signalling that they have not used 3.12, and that they assume their pre-3.12 mental model is current.

What saying it correctly tells them: You have read PEP 701 or you have hit this in production. Either is fine. Either makes you look current.

1.3 Per-interpreter GIL groundwork (PEP 684)

This is the lowest-volume of the three 3.12 features but it is the one that opens the door to the much bigger 3.13 free-threaded conversation, so it gets asked at the senior end as a setup for that follow-up.

PEP 684 landed the C-level work to give each subinterpreter its own GIL. It is not yet a fully ergonomic Python-level feature — the public `interpreters` module remains in `_xxsubinterpreters` form in 3.12 — but the groundwork is in place. PEP 734 is proposed for 3.14 and will expose the public `concurrent.interpreters` module; 3.13 still exposes the machinery only privately (`_interpreters`).

For interview purposes in 2026, what matters is awareness of the direction. A senior candidate should be able to say something like: "3.12 landed the per-interpreter GIL at the C level (PEP 684), which means each subinterpreter has its own GIL and can run in parallel on separate OS threads. The Python-level API to use this is still maturing — the `_xxsubinterpreters` module in 3.12 is private; 3.13 moved it forward. This is one of two parallel paths CPython is exploring for parallelism — the other being the 3.13 free-threaded build under PEP 703."

How interviewers ask this: As a follow-up to the GIL question. "Given what we know about the GIL, are there any directions CPython is exploring to relax it?" A junior candidate stops at "they are working on it." A mid-level mentions free-threading. A senior mentions both per-interpreter GIL (PEP 684) and free-threading (PEP 703) and articulates why CPython is pursuing two parallel paths.

What saying it correctly tells them: You read PEPs. You are not just a Python user; you follow the language design conversation.

A minimal example of what the subinterpreter machinery looks like in 3.12 (note the leading underscore — this is a private API):

```
# 3.12: the private API. Use at your own risk; the public API is still in flux.
import _xxsubinterpreters as interpreters

interp_id = interpreters.create()
interpreters.run_string(interp_id, "print('hello from subinterpreter')")
interpreters.destroy(interp_id)
```

PEP 734 is proposed for 3.14 and will expose the public `concurrent.interpreters` module; 3.13 still exposes the machinery only privately (`_interpreters`). Mentioning the rename trajectory is the senior-tier flourish.

1.4 Other 3.12 features worth knowing for completeness

These come up less often but show up in scattered rounds:

- **typing.override decorator (PEP 698)** — covered in detail in Section 4 below. Don't conflate this with C#'s `override` — no runtime enforcement; only attaches an `__override__` attribute that static checkers and runtime tools can read.
- **Improved error messages** — interviewers don't ask about these directly, but they affect the "guess the output" traps. A `NameError` now suggests likely typos; a `SyntaxError` on f-strings is now precise.

- `pathlib.Path.walk()` — a clean replacement for `os.walk` that returns `Path` objects. Mid-level Backend rounds occasionally probe this when the topic is file I/O.
- `itertools.batched` — landed in 3.12, batches an iterable into fixed-size tuples. Showing up in rounds where the interviewer asks for "chunk a list into groups of N" — the 2026 answer reaches for `batched` rather than the classic `range(0, len(...), N)` slice pattern.

```
from itertools import batched

for chunk in batched(range(10), 3):
    print(chunk)
# (0, 1, 2)
# (3, 4, 5)
# (6, 7, 8)
# (9,)
```

A candidate who reaches for `itertools.batched` for the chunking question — rather than the slice pattern or the `more_itertools` import — signals they are on 3.12 in production.

Section 2 — Python 3.13 features (early signals)

3.13 shipped in October 2024. By mid-2026 it has had about eighteen months to propagate, but the two headline features — the free-threaded build and the experimental JIT — are still gated behind opt-in flags and shipped as experimental. Indian production stacks are not yet running 3.13 in any meaningful volume; Razorpay and Flipkart have it in dev environments for evaluation; Service MNC client work is universally on 3.10 to 3.12.

That gradient determines how 3.13 is asked about in 2026 interviews. The interviewer is checking whether you have heard of these features and can articulate them — they are not checking whether you have shipped them to production. The right framing for the candidate is "I know what these are, I know they are experimental, here is what they change conceptually." Pretending to have shipped 3.13 free-threaded code to production will get you caught.

2.1 The free-threaded build — PEP 703 and the no-GIL CPython

This is the single most exciting Python development of the decade and interviewers know it. PEP 703 proposes — and 3.13 ships as experimental — a build of CPython without the GIL. The build is opt-in at compile time (`./configure --disable-gil`) and the binary is distinct from the standard CPython binary. PyPI packages with C extensions need explicit compatibility work; a large fraction of the scientific Python ecosystem (NumPy, SciPy, pandas) has begun shipping free-threaded wheels but the long tail of packages has not.

For interview purposes the key claims to be able to make confidently:

- PEP 703 disables the GIL by replacing it with finer-grained locking — primarily per-object locks plus biased reference counting modifications.
- The free-threaded build is experimental in 3.13 and 3.14. The official roadmap targets "supported but not default" in 3.15 and "default" later.
- Single-threaded performance of the free-threaded build is somewhat worse than the standard build — early benchmarks showed 5-10% regression, since narrowed.
- C extensions need to declare free-threaded compatibility via a module-level flag. Extensions that have not been audited fall back to a per-module GIL.

The candidate-side framing that lands well: "I know it exists, I know it is experimental, I know it changes the threading-vs-multiprocessing-vs-asyncio reasoning if it ships, I am not running it in production yet."

```
# 3.13 free-threaded build – check at runtime whether the GIL is disabled
import sys

if hasattr(sys, "_is_gil_enabled"):
    print(f"GIL enabled: {sys._is_gil_enabled()}")
else:
    print("Pre-3.13 – GIL is always enabled")
```

How interviewers ask this: As a senior follow-up to the GIL question. "Given the standard GIL story, are there any developments that would change your answer?" The expected response is to bring up free-threaded 3.13 with appropriate hedging. A candidate who has not heard of it loses the senior tag immediately.

2.2 The experimental JIT — copy-and-patch JIT compilation

3.13 shipped an experimental JIT compiler under PEP 744 work, using the "copy-and-patch" technique pioneered by academic compilers. It is opt-in at build time (`./configure --enable-experimental-jit`) and disabled by default. Early performance numbers were modest — a few percent improvement on representative workloads — and the JIT is positioned as the foundation for more aggressive optimization in later releases.

For interview purposes the key facts to have at hand:

- 3.13's JIT uses the copy-and-patch technique. The JIT runs after the existing tier-1 (specialized bytecode) interpreter and produces machine code for hot bytecode regions.
- It is opt-in, experimental, and produces modest gains in 3.13. The serious gains are expected in 3.14 and beyond.
- It is distinct from earlier Python JIT efforts like PyPy (a full alternative implementation) and Numba (a domain-specific JIT for numerical code).

How interviewers ask this: Rarely directly. More commonly as part of a "what is the future of Python performance" senior question. The candidate's job is to mention the JIT, mention that it is experimental, and not overstate its current impact.

2.3 Incremental cycle GC

3.13 made Python's cyclic garbage collector incremental — generations are still preserved, but the GC's work per cycle is broken into smaller chunks, which reduces the worst-case pause time. This is the kind of change that matters in production but is not by itself an interview question. Where it shows up: as a follow-up to "how does Python's GC work" at the senior Backend end. Mentioning that 3.13 made the cycle collector incremental is a credible signal.

The base answer for the GC question has not changed: CPython uses reference counting as the primary mechanism (each object has a refcount, deallocation is immediate when it drops to zero), with a backup cyclic collector to handle reference cycles. The 3.13 change is at the cyclic collector layer.

2.4 Removed dead modules

3.13 removed a batch of modules that had been deprecated for years — `aifc`, `audioop`, `cgi`, `cgitb`, `chunk`, `crypt`, `imghdr`, `mailcap`, `msilib`, `nis`, `nntplib`, `ossaudiodev`, `pipes`, `sndhdr`, `spwd`, `sunau`, `telnetlib`, `uu`, `xdrlib`. The removal is collectively known as PEP 594.

For interview purposes this is not asked directly but it is asked indirectly: a candidate who reaches for `cgi` to parse a multipart form, or who suggests `telnetlib` for any reason, signals they have not opened a release notes page in years. The expected 2026 answer for multipart form parsing is one of `email`, `multipart`, or framework-provided (FastAPI's `Form`, Django's request parsing).

2.5 Other 3.13 items worth flagging

- The new REPL — 3.13 replaced the venerable Python interactive prompt with a vastly better one (multi-line editing, history search, colour output, paste mode). Interviewers don't ask about this but candidates who do live coding in a 3.13 REPL look polished.
- `typing.TypeIs` (PEP 742) — a narrowing alternative to `TypeGuard` with cleaner semantics. Niche but climbing.
- `argparse` got the `deprecated=True` keyword on arguments — useful for CLI migrations.

The single most important thing to internalize about 3.13: in mid-2026 it is "have you heard of these" material, not "have you shipped these" material. Frame your answers accordingly.

Section 3 — Async patterns in 2026

The async story in Python has firmed up considerably in the last two years. Before 3.11, `asyncio` was a library that you used with `gather`, `wait`, `create_task`, and a lot of defensive error handling. After 3.11 — with `TaskGroup`, `asyncio.timeout`, and the structured-concurrency vocabulary that PEP 654 enabled — async code is starting to look like async code in Trio: structured, hierarchical, with predictable error propagation.

Interviewers have caught up. The 2026 async question is no longer "what is async/await" — that is screening-round material. The 2026 async question is "given a structured set of async operations, what primitive do you reach for, and what changes when one of them fails."

3.1 `asyncio.TaskGroup` — the structured concurrency primitive

Added in 3.11, `TaskGroup` is now the right answer for "run these N coroutines concurrently and wait for all of them." It supersedes `asyncio.gather` for almost every use case.

The conceptual contrast is worth being able to articulate. `asyncio.gather` is a function that returns a future containing the results of N awaitables. `TaskGroup` is an async context manager that owns the tasks created inside it. When the `async with TaskGroup()` block exits normally, all tasks are awaited. When one task raises, the others are cancelled and the exception is propagated.

This propagation behaviour is the headline win. With `gather(return_exceptions=False)`, if one task fails the others continue running until they finish naturally, and you have to write defensive cleanup code. With `TaskGroup`, the failure of one task automatically cancels the siblings, and exceptions are bundled into an `ExceptionGroup` (PEP 654) at the boundary.

```
import asyncio

async def fetch(url: str) -> str:
    # ... pretend this hits a network ...
    await asyncio.sleep(0.1)
    return url

async def fetch_all(urls: list[str]) -> list[str]:
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(fetch(url)) for url in urls]
    # If we reach here, all tasks finished without exception.
    # If any task raised, control never gets here — the ExceptionGroup
    # is raised at the end of the `async with` block.
    return [t.result() for t in tasks]
```

The `gather`-flavoured equivalent that the same candidate would have written in 2023:

```
async def fetch_all_gather(urls: list[str]) -> list[str]:
    return await asyncio.gather(*(fetch(url) for url in urls))
```

The two look equivalent for the happy path. They diverge sharply when something fails.

How interviewers ask this: The standard probe in 2026 Backend rounds is "you have ten HTTP calls to make in parallel — how do you write that?" A candidate who writes `gather` is asked the follow-up: "what happens if one of them fails halfway through?" If the candidate cannot articulate that `gather` leaves the siblings running and that `TaskGroup` cancels them with `ExceptionGroup` propagation, they get marked down. The interviewer is checking whether the candidate has written async code in the last eighteen months or whether they learned async in 2021 and stopped.

What saying it correctly tells them: You have read PEP 654, or you have hit the gather-cleanup pain in production and migrated. Either signals current.

3.2 `asyncio.timeout` — the context-manager timeout

Added in 3.11, `asyncio.timeout` is the structured-concurrency-flavour replacement for `asyncio.wait_for`. The shape is a context manager rather than a wrapping function, which composes much better with `TaskGroup` and with the rest of the async code.

```
import asyncio

async def fetch_with_timeout(url: str) -> str:
    try:
        async with asyncio.timeout(5): # 5-second budget
            return await fetch(url)
    except TimeoutError:
        return ""
```

The contrast with the older `wait_for` shape:

```
async def fetch_with_timeout_old(url: str) -> str:
    try:
        return await asyncio.wait_for(fetch(url), timeout=5)
    except TimeoutError:
        return ""
```

The `wait_for` form wraps a single awaitable. The `timeout` form wraps a region — anything inside the `async with` block is subject to the deadline. This composes far better when you have nested operations.

There is also `asyncio.timeout_at(deadline)` for absolute (rather than relative) deadlines, useful when a budget is set at the top of a request and individual operations need to respect the same wall-clock deadline.

```
import asyncio
import time

async def handle_request(req):
    deadline = asyncio.get_running_loop().time() + 30 # 30s overall budget
    async with asyncio.timeout_at(deadline):
        await step_one(req)
        await step_two(req)
        await step_three(req)
```

How interviewers ask this: As a follow-up to the `TaskGroup` question. "Now add a 5-second deadline to that fetch group." A candidate who reaches for `wait_for` is signalling pre-3.11 patterns. A candidate who reaches for `asyncio.timeout` (and ideally mentions `timeout_at` for the absolute-deadline case) signals current.

3.3 `TaskGroup` + `timeout` composed together

The two primitives compose cleanly, and the composed pattern is the actual 2026 answer that interviewers want to see when the question is "concurrent fetches with a total deadline."

```
import asyncio

async def fetch(url: str) -> str:
    await asyncio.sleep(0.5)
    return url

async def fetch_all_with_deadline(urls: list[str], deadline_sec: float) -> list[str]:
    async with asyncio.timeout(deadline_sec):
        async with asyncio.TaskGroup() as tg:
            tasks = [tg.create_task(fetch(url)) for url in urls]
        return [t.result() for t in tasks]
```

If the deadline elapses, the `timeout` cancels everything inside it, including the `TaskGroup`'s outstanding tasks, and a `TimeoutError` is raised. This is the structured-concurrency story end-to-end: composition, predictable cancellation, no leaked tasks.

3.4 `ExceptionGroup` and `except*` (PEP 654)

`TaskGroup` raises `ExceptionGroup` rather than a single exception when multiple tasks fail. PEP 654 added the `except*` syntax to handle these.

```
async def main():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(may_raise_value_error())
            tg.create_task(may_raise_runtime_error())
    except* ValueError as eg:
        for exc in eg.exceptions:
            print(f"value error: {exc}")
    except* RuntimeError as eg:
        for exc in eg.exceptions:
            print(f"runtime error: {exc}")
```

Note the `except*` form (not `except`). The `*` means "match any exception in the group of this type, and let the rest re-raise." This is genuinely new syntax — 3.11+ only — and writing it correctly signals current.

The common candidate mistake is to write `except ExceptionGroup as eg` and then iterate `eg.exceptions` manually. That works but it loses the routing-by-type behaviour that `except*` gives you. The interviewer will probe this distinction.

3.5 When NOT to use async

This is the part most candidates get wrong in 2026 rounds, and it is what separates a candidate who knows async syntax from a candidate who knows when to reach for it.

The rule of thumb that lands well in interviews:

- **Async** is for I/O-bound concurrency where you have many concurrent operations and the framework (web server, HTTP client) already supports it. Web servers (FastAPI, Starlette), database clients with async drivers (`asyncpg` , `aiomysql`), HTTP clients (`httpx.AsyncClient`), and message broker clients are the canonical homes.
- **Threading** is for I/O-bound concurrency where the underlying library does not have an async version and you cannot or will not rewrite it. Most legacy libraries that block on sockets — older HTTP clients, some database drivers — fit here. Threading also works when you have a small fixed number of concurrent operations and don't want async's structural cost.
- **Multiprocessing** is for CPU-bound work that fits a worker-pool model. The GIL releases under multiprocessing because each process has its own. ML feature pipelines, image processing, and parallel numerical work fit here.

A candidate who says "we should always use async because it is more efficient" has misunderstood. Async only helps if the operations you are running are themselves async-aware. Async fronting a synchronous database driver gives you nothing.

The follow-up question that catches candidates: "you have a Python service that needs to call a third-party HTTP API 100 times per request. The third-party only has a synchronous SDK. What do you do?" The 2026 right answer is one of:

- Wrap each call in `asyncio.to_thread(sdk.call, args)` — this offloads the synchronous call to the default thread pool, allowing your async event loop to continue handling other work. This is the cleanest 3.9+ pattern.
- Use a `ThreadPoolExecutor` with `loop.run_in_executor(pool, sdk.call, args)` — older but still valid.
- Use a thread pool with `concurrent.futures.ThreadPoolExecutor` and a synchronous web framework. This is the right answer if the whole stack is synchronous.

```
import asyncio

def sync_sdk_call(url: str) -> str:
    # ... blocking SDK ...
    return url

async def call_many(urls: list[str]) -> list[str]:
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(asyncio.to_thread(sync_sdk_call, url)) for url in urls]
    return [t.result() for t in tasks]
```

How interviewers ask this: As the third or fourth question in an async sequence. "Now imagine the third-party SDK is synchronous — does your answer change?" The candidate who reaches for `to_thread` and articulates why is signalling current. The candidate who suggests "rewrite the SDK to be async" is signalling textbook understanding without production experience.

3.6 The standard async interview probe in 2026

The canonical async probe at Razorpay, PhonePe, Postman, Chargebee, and most GCC FastAPI shops in 2026 is some variation of:

1. Write a function that fetches N URLs concurrently.
2. Now add a 5-second total deadline.
3. Now handle the case where one or more fetches fail without bringing down the others.
4. Now imagine the underlying client library is synchronous — what changes?
5. When would you reach for threading instead of asyncio for this?

The candidate who writes `TaskGroup` + `asyncio.timeout` + `except*` for the first three, `asyncio.to_thread` for the fourth, and articulates the threading-vs-async tradeoff for the fifth has passed the async section. Anything less than this is a clear marker for "did not write Python in 2024-2026."

Section 4 — Typing updates

The typing landscape has shifted more between 2022 and 2026 than in any comparable two-year window since type hints were introduced. The cumulative effect is that typing fluency has crossed from "nice to have" to "how seniors signal seniority" in Backend interviews at Indian product cos and GCCs. A candidate who can write a correctly-typed decorator with `ParamSpec` and `Concatenate`, or who reaches for `Self` and `@override` reflexively, is doing in 2026 what writing list comprehensions correctly did in 2018: signalling that they have absorbed the language's recent direction.

Service MNC rounds lag here. TCS and Infosys mid-level rounds rarely probe `ParamSpec` or `Concatenate`. They will probe `Optional` vs `None`, `Union` vs the `|` syntax, and `TypedDict` at most. The deeper typing material is concentrated in Indian product co Backend rounds and GCC FastAPI/Django rounds.

There are five typing topics that are now interview-table-stakes in 2026 senior Backend rounds. Each gets its own treatment below.

4.1 The `@override` decorator (PEP 698 / 3.12)

The `typing.override` decorator is new in 3.12 and it is the single most-asked typing addition of the last twelve months. It marks a method as deliberately overriding a parent class's method; the static type checker (mypy, pyright) then verifies that the override is real.

The motivating problem: in Python, if you misspell an override, the type checker has no way to know you intended to override. The misspelled method silently becomes a new method on the subclass, the parent method continues to be called, and the bug is hard to spot.

```
from typing import override

class Animal:
    def make_sound(self) -> str:
        return "generic noise"

class Dog(Animal):
    @override
    def make_sound(self) -> str: # mypy verifies that Animal has make_sound
        return "woof"

class Cat(Animal):
    @override
    def maek_sound(self) -> str: # typo - mypy flags this
        return "meow"
```

The decorator is a pure type-checker hint at runtime — calling `@override` has no behavioural effect. It exists to give the type checker a hook to verify intent.

The closest parallel from other languages is C#'s `override` keyword and Java's `@Override` annotation. The Python version is closer to Java's `@Override` (annotation, not keyword) and it serves the same purpose: catch the bug where the developer intended to override but misnamed the method.

How interviewers ask this: "How would you catch the bug where I intend to override a method but I misspell the name?" The expected 2026 answer is `@override` from `typing`. Bonus points for explaining that it is a type-checker hint, not a runtime check, and that it requires Python 3.12 or — for older Pythons — the `typing_extensions` backport.

What saying it correctly tells them: You have read PEP 698 or you have used `@override` in production. You know that the bug it catches is real. You are following the typing direction.

A senior follow-up: "what if I want this in 3.11?" The answer is `from typing_extensions import override`, which has been backporting these for years.

4.2 `TypeAliasType` and the `type` statement

Covered briefly in Section 1.1. Worth expanding on the typing-side semantics because they come up at the senior end.

The `type` statement creates a `TypeAliasType` object. The right-hand side is evaluated lazily, which is meaningful for forward references and for self-referential types like trees or recursive JSON structures.

```
type JSON = dict[str, "JSON" | list["JSON"] | str | int | float | bool | None]

# Before 3.12 – the equivalent required explicit string quoting and a TypeAlias annotation
from typing import TypeAlias

JSONOld: TypeAlias = "dict[str, JSONOld] | list[JSONOld] | str | int | float | bool | None"
```

The `TypeAliasType` form is cleaner, lazier, and works in more places without quoting.

How interviewers ask this: "Define a recursive JSON type in Python." The 2024 answer was the `TypeAlias` form with manual string quoting. The 2026 answer is the `type` statement. A candidate who writes the new form correctly without quoting the recursive reference is signalling current.

4.3 `ParamSpec` (PEP 612 / 3.10) and `Concatenate`

`ParamSpec` shipped in 3.10 but only became interview-mainstream in 2024–2025 as decorator-typing patterns crystallized. It is the right answer for "type a decorator that wraps any callable while preserving its signature."

The problem `ParamSpec` solves: before it existed, you could not write a decorator type that preserved the wrapped function's parameter signature. You could write `Callable[..., R]` for the return type and lose all parameter typing, or you could write a bespoke `TypeVar`-based shape that only worked for one specific signature.


```

from typing import Callable, ParamSpec, TypeVar
import functools
import time

P = ParamSpec("P")
R = TypeVar("R")

def timed(fn: Callable[P, R]) -> Callable[P, R]:
    @functools.wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            elapsed = time.perf_counter() - start
            print(f"{fn.__name__} took {elapsed:.3f}s")
    return wrapper

@timed
def add(x: int, y: int) -> int:
    return x + y

add(1, 2) # type-checks; the type checker knows add still takes two ints

```

Concatenate is a sibling primitive that lets you write decorators which add or remove leading parameters. The canonical use case is the `@classmethod`-flavoured decorator that consumes the first argument:

```

from typing import Callable, ParamSpec, TypeVar, Concatenate

P = ParamSpec("P")
R = TypeVar("R")

def with_user(fn: Callable[Concatenate[str, P], R]) -> Callable[P, R]:
    """A decorator that injects the current user as the first argument."""
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        return fn(current_user(), *args, **kwargs)
    return wrapper

def current_user() -> str:
    return "abhishek"

@with_user
def greet(user: str, greeting: str) -> str:
    return f"{greeting}, {user}"

greet("hello") # the type checker knows greet now takes only the greeting

```

How interviewers ask this: "Type a decorator that times a function." A 2022 candidate would write `Callable[..., Any]` and the interviewer would flag it. A 2026 candidate writes `ParamSpec` and

gets the senior tag. A senior follow-up is to add a leading argument with `Concatenate` — this is the "OK now I want it to also inject the user" question.

What saying it correctly tells them: You have written real decorators with real typing. `ParamSpec` and `Concatenate` are exactly the things you reach for when you have a non-trivial wrapping problem, and reaching for them naturally is the senior signal.

4.4 The `Self` type (PEP 673 / 3.11)

`Self` shipped in 3.11. It is the type you reach for when a method returns "this same type" — which is most commonly the case in builder patterns and in `from_*` classmethods.

```
from typing import Self
from dataclasses import dataclass

@dataclass
class QueryBuilder:
    _filters: list[str]
    _limit: int | None = None

    def where(self, clause: str) -> Self:
        return type(self)(_filters=self._filters + [clause], _limit=self._limit)

    def limit(self, n: int) -> Self:
        return type(self)(_filters=self._filters, _limit=n)

    @classmethod
    def for_table(cls, name: str) -> Self:
        return cls(_filters=[f"FROM {name}"])
```

The `Self` annotation tells the type checker that `where` returns the same subclass it was called on. If you subclass `QueryBuilder` and call `where`, the type checker knows you get back the subclass, not the base.

Before `Self`, the equivalent required a `TypeVar` bound to the class:

```
from typing import TypeVar

T = TypeVar("T", bound="QueryBuilder")

class QueryBuilderOld:
    def where(self: T, clause: str) -> T:
        ...
```

The `Self` form is cleaner and is what the type checkers now optimize for.

How interviewers ask this: "How would you type a fluent builder?" The 2022 answer is the bound `TypeVar`. The 2026 answer is `Self`. A candidate who reaches for the bound `TypeVar` is not wrong, but they are signalling pre-3.11 patterns.

4.5 Pydantic v2 — the migration that finished

Pydantic v2 was released in mid-2023. By mid-2026 the v1 to v2 migration arc is finished — v2 is the default assumption in any Backend interview that touches FastAPI or any service that does data validation. The interviewers know this and they probe it as a proxy for "have you shipped Python services in the last two years."

The most-asked Pydantic-related questions in 2026 Backend rounds:

4.5.1 The `model_config` switch

In v1, model configuration was done via a nested `class Config:` declaration. In v2, the same configuration is done via a `model_config = ConfigDict(...)` class attribute. This change is the single most visible v1-to-v2 difference and it is the first thing interviewers probe.

```
# Pydantic v1
from pydantic import BaseModel # v1 import

class UserV1(BaseModel):
    name: str
    email: str

    class Config:
        anystr_strip_whitespace = True
        validate_assignment = True

# Pydantic v2
from pydantic import BaseModel, ConfigDict # v2 import (same path, different class behaviour)

class UserV2(BaseModel):
    model_config = ConfigDict(
        str_strip_whitespace=True, # renamed from anystr_strip_whitespace
        validate_assignment=True,
    )

    name: str
    email: str
```

Note both the configuration syntax change and the keyword rename (`anystr_strip_whitespace` to `str_strip_whitespace`). Interviewers love the rename detail because it filters candidates who have actually migrated from candidates who have only read about the change.

4.5.2 Validators

v1's `@validator` and `@root_validator` are replaced in v2 by `@field_validator` and `@model_validator`. The signatures differ. The `mode` keyword (`"before"`, `"after"`, `"wrap"`) replaces the v1 `pre` and `each_item` flags.

```
from pydantic import BaseModel, field_validator, model_validator

class Order(BaseModel):
    amount: float
    currency: str

    @field_validator("currency")
    @classmethod
    def normalize_currency(cls, v: str) -> str:
        return v.upper()

    @model_validator(mode="after")
    def check_amount_positive(self) -> "Order":
        if self.amount <= 0:
            raise ValueError("amount must be positive")
        return self
```

Note that v2 `@field_validator` is a `classmethod` and the validator's `cls` parameter is the first argument. The v2 `@model_validator` with `mode="after"` operates on the constructed instance — the `self` first argument.

4.5.3 Performance

v2's core is implemented in Rust (the `pydantic-core` package). Performance is materially better than v1 — most workloads see 5-50x speedup depending on shape. Interviewers ask about this as a "why does v2 exist" question. The expected answer mentions both the Rust core and the API redesign that the rewrite enabled.

4.5.4 The `dataclass` story

v1 had `pydantic.dataclasses.dataclass`. v2 retained it but added `BaseModel`-flavoured equivalents. The right 2026 advice: use `BaseModel` unless you specifically need a dataclass that integrates with code expecting `dataclasses.is_dataclass`.

How interviewers ask all of this: The canonical 2026 Pydantic question is some variant of "what changed between v1 and v2 — what did you have to migrate?" The interviewer is checking whether the candidate has actually done the migration. A candidate who knows only the `class Config:` to `model_config` switch and stops is signalling "I read about v2 but did not migrate." A candidate who can cite the validator rename, the `mode` keyword, the Rust core, and at least one specific renamed config key is signalling they shipped a v2 migration in the last eighteen months.

What saying it correctly tells them: You have shipped production Python services in 2024-2026. You hit the migration. You know the gotchas. You are senior.

4.6 Other typing items worth knowing

For completeness, briefly:

- `from __future__ import annotations` — the magic import that defers annotation evaluation. Still useful in 3.10/3.11 but less so in 3.12+. PEP 649 (deferred annotation evaluation as the default) targets 3.14. The 2026 right answer for typing questions in 3.12 code is to not need the `__future__` import.
- `TypedDict` — established. Use it for dict-shaped data when you don't want a class. The `Required` and `NotRequired` markers (PEP 655 / 3.11) let you mark individual keys as optional.
- `Unpack` (PEP 646, 692) — unpacks a `TypedDict` into `**kwargs`. Niche but climbing — used for typing keyword arguments that mirror a config dict.
- `Protocol` — structural typing. Use it when you want duck-typed interfaces without nominal inheritance. Established and frequently asked.
- `Final` — marks a class attribute as constant. Use it for true constants. Asked as a "what does Final mean" question at the mid-senior boundary.
- `Literal` — restricts a parameter to a fixed set of values. Used heavily in API design and config validation.

A senior candidate in 2026 Backend rounds should be able to combine these. The composite question "type this configuration loader" — where the answer involves `TypedDict` with `Required` / `NotRequired`, `Literal` for enum-like fields, and possibly `Protocol` for plugin interfaces — is the high-end typing probe.

Section 5 — pandas 2.x patterns

The pandas interview question diet has shifted more than any other Data-track topic in the last twelve months. pandas 2.0 shipped in April 2023; by 2.2 (early 2024) the Arrow backend was production-ready and the copy-on-write semantics were stabilized. By mid-2026, pandas 2.x is the default assumption in Data-role interviews at Indian product cos and GCCs. Candidates who only know pandas 1.x patterns are flagged within the first ten minutes.

Service MNC client work still runs on pandas 1.x in places where the data platform was built before 2023 and has not been migrated. Indian product cos like Flipkart, Swiggy, Meesho, and Razorpay's data team are on 2.x. GCCs are mixed — newer projects on 2.x, legacy on 1.x. The interview question gradient maps onto this.

There are four pandas 2.x topics that are now interview-table-stakes in 2026 Data-role rounds. Each gets its own treatment.

5.1 The Arrow backend

pandas 2.0 introduced the option to use Apache Arrow as the storage backend instead of NumPy. The shape of the API stayed the same; the underlying memory layout changed. The interview-relevant consequences:

- Native nullable types for all dtypes. No more "int columns can't hold NaN so they get promoted to float."
- String operations are faster — orders of magnitude on large columns, because Arrow's string layout is contiguous and the operations are vectorized at the C level.
- Categorical-like behaviour without the explicit dtype.
- Memory layout matches what Parquet and other columnar formats use natively — read/write is faster and cheaper.

The interview probe is whether the candidate knows how to opt into the Arrow backend.

```
import pandas as pd

# Reading CSV with Arrow-backed dtypes
df = pd.read_csv("orders.csv", dtype_backend="pyarrow")

# Or convert an existing frame
df_arrow = df.convert_dtypes(dtype_backend="pyarrow")
```

The `dtype_backend="pyarrow"` keyword is the right-answer detail. The legacy `dtype_backend="numpy_nullable"` also exists for "NumPy with masked arrays" — useful as a middle ground but not the headline 2.x feature.

A consequence the candidate should be able to articulate: under Arrow, `pd.NA` is the universal missing-value marker. Under NumPy, you get `np.nan` for floats, `None` for objects, and `NaT` for datetimes. The Arrow backend collapses this into one consistent missing-value type, which removes a whole class of "why is this filter not matching" bugs.

The concrete numbers the candidate should have at hand for the senior end of this question. String operations under Arrow are typically 3-10x faster on large columns (10M+ rows) for `.str.contains`, `.str.lower`, `.str.split` — the contiguous string layout means the work runs in a tight C loop instead of dereferencing a Python object per row. Memory footprint for string-heavy frames typically drops 30-50% because Arrow's dictionary encoding compresses repeated values automatically. Read times for Parquet files improve by 2-4x because the Arrow in-memory layout matches Parquet's on-disk layout — there is no per-row conversion step. The Arrow CSV parser (`engine="pyarrow"` on `read_csv`) is roughly 5-10x faster than pandas's native parser on 1 GB+

files. A candidate who can quote two or three of these numbers, even approximately, signals they have benchmarked Arrow on real data rather than just read the blog post.

How interviewers ask this: "How do you load this 5 GB CSV efficiently?" The 2026 answer mentions `dtype_backend="pyarrow"`, `engine="pyarrow"` (the Arrow C++ CSV parser is materially faster than pandas's native one), and possibly chunked reads if memory is the bottleneck. A candidate who only mentions `chunksizes=` is signalling pre-2.x patterns.

What saying it correctly tells them: You have worked with real-size data in pandas 2.x. You know the Arrow story. You are likely a Data Engineer or Data Scientist who has shipped pipelines, not just done coursework.

5.2 Copy-on-write semantics

This is the single most-asked pandas 2.x interview topic in 2026, and the one candidates most often get half-right. pandas 2.x has been opting into copy-on-write (CoW) semantics in stages — opt-in in 2.0, stabilized in 2.2, default in 3.0 (targeted for 2024 but slipped; expected in a future major release, timeline still moving).

The core issue CoW solves: pandas has historically been inconsistent about whether a derived DataFrame is a view onto the original or a copy. The infamous `SettingWithCopyWarning` exists because the answer is sometimes "view," sometimes "copy," and the user can't tell without inspecting internals. Code that "works" can silently break when an upstream operation changes from returning a view to returning a copy.

CoW gives pandas a consistent model: derived frames always behave as if they are independent. Internally, pandas may still share memory until a write occurs, at which point it copies. To the user, derived frames look completely independent — which means `SettingWithCopyWarning` disappears, and `chained assignment` either works deterministically or fails clearly.

```
import pandas as pd

# Enable CoW in 2.x
pd.set_option("mode.copy_on_write", True)

df = pd.DataFrame({"a": [1, 2, 3], "b": [10, 20, 30]})
sub = df[df["a"] > 1]
sub["b"] = 99 # under CoW, this never modifies df — guaranteed

# Without CoW, the same code may or may not modify df depending on internals;
# pandas raises SettingWithCopyWarning to let you know it might.
```

The right candidate answer here is to know that CoW is opt-in in 2.x today and will be the default in 3.0. The migration question — "if I have a pandas 1.x codebase that relied on view semantics, what breaks under CoW" — comes up at the senior end.

The concrete migration patterns that break under CoW, in priority order. First, any code that relied on chained-assignment side effects: `df[df["a"] > 0]["b"] = 99` no longer modifies `df` under CoW — the assignment lands on a transient slice that is then discarded. The fix is to write `df.loc[df["a"] > 0, "b"] = 99` and stop relying on the implicit chain. Second, any code that grabbed a column via `col = df["a"]` and then mutated `col` in place expecting `df` to reflect the change — under CoW the column is independent. Third, the `inplace=True` keyword has been deprecated in spirit for years and CoW is what finally makes the warnings sharp; the migration is to return-and-reassign (`df = df.drop(...)`) rather than to mutate. Fourth, code that grabbed a slice via `.iloc[:5]` and then expected mutations to that slice to propagate — they do not. The migration playbook for an existing 1.x codebase is to turn CoW on early in a staging environment, run the test suite, and fix every `SettingWithCopyWarning` -flavoured failure one by one. Most of the fixes are mechanical.

The other half of the CoW story is that `SettingWithCopyWarning` itself is on the path to becoming a hard error rather than a warning under CoW defaults. The 2.x story is "warning today, behaviour change tomorrow." The 3.0 story is "the ambiguous case is no longer ambiguous — the operation either succeeds deterministically or fails loudly." A candidate who frames the warning as a deprecation path rather than a runtime annoyance is signalling they understand where the library is going.

How interviewers ask this: "Have you seen `SettingWithCopyWarning` ?" — and then the follow-up: "What is changing in pandas 3.0 to address this?" The 2026 answer is CoW. A candidate who only says "use `.loc` to set" is giving a 1.x-era answer.

What saying it correctly tells them: You have followed pandas's direction. You know the 3.0 plan. You probably hit the bug yourself and have the scar tissue.

5.3 `pd.NA` vs `np.nan` — the missing-value distinction

This is the second most-asked pandas 2.x topic in 2026 and it is mostly asked as a Guess-the-Output trap. The key distinction:

- `np.nan` is a float. It is `nan` in IEEE 754 sense. It does not equal itself: `np.nan == np.nan` is `False`. It propagates in arithmetic: `np.nan + 1 == nan`. It propagates in comparisons: `np.nan > 0 == False`, `np.nan < 0 == False`. Used historically for any "missing" value in pandas, even in integer columns (which had to be promoted to float).
- `pd.NA` is the "true" missing-value marker introduced in pandas 1.0 and made universal in 2.x with the Arrow backend. It is logical-three-valued: comparisons return `pd.NA` rather than `True` or `False`. Arithmetic with `pd.NA` returns `pd.NA`. Boolean operations follow Kleene logic: `pd.NA | True` is `True`, `pd.NA | False` is `pd.NA`.


```
import pandas as pd
import numpy as np

# Under Arrow / nullable dtypes, missing is pd.NA
s = pd.Series([1, 2, None], dtype="Int64") # capital I – nullable
print(s)
# 0      1
# 1      2
# 2    <NA>
# dtype: Int64

# Boolean operations follow three-valued logic
print(pd.NA | True) # True
print(pd.NA | False) # <NA>
print(pd.NA & False) # False
print(pd.NA & True) # <NA>
```

The interview trap: candidates who try to filter using `df[df["col"] == pd.NA]` get an empty result because the comparison returns `pd.NA`, which is falsy. The right pattern is `df[df["col"].isna()]`. This is the same trap as `WHERE col = NULL` in SQL.

How interviewers ask this: As a Guess-the-Output trap or as a "filter the rows where this column is missing" probe. The 2026 right answer reaches for `.isna()` reflexively. A candidate who tries `== np.nan` or `== pd.NA` and is confused by the result is signalling they have not used pandas 2.x extensively.

What saying it correctly tells them: You know the missing-value model. You have hit the trap and remembered.

A senior follow-up that has been surfacing in 2026 Data rounds at Razorpay and Postman: "how does this compare to how Pydantic v2 handles optional fields?" The framing the candidate is meant to land. Pydantic v2 distinguishes `None` (an explicit null) from "field not provided" (defaulted) via the `Optional[T]` vs `T = None` distinction, and validation runs once at construction time. pandas with `pd.NA` distinguishes the missing value from a sentinel `None` only when the dtype is a nullable extension type — under Arrow or `"Int64" / "Float64" / "string"`, `pd.NA` is the missing marker; under the default NumPy backend, missing collapses back into `np.nan` or `None` depending on the column type. The deeper comparison: Pydantic validates eagerly and rejects bad data at the boundary; pandas validates lazily and keeps the bad data, leaving you to discover it via `.isna()` checks or aggregation surprises. A candidate who can articulate this difference signals they have shipped data validation in both worlds.

5.4 The `apply` performance cliff

This is not new to pandas 2.x — it has been the case forever — but it is being probed harder in 2026 because Data interview question banks have shifted toward performance reasoning. The question is "when does `apply` fall off a cliff, and how do you recognize it before you ship the slow code."

The mechanics: `df.apply(fn, axis=1)` calls `fn` once per row. The per-row overhead is large — there is a Python-level function call and a Series construction for each row. For a million-row DataFrame, this is roughly a million Python-level calls. Equivalent vectorized operations are typically 100-1000x faster because they push the work into NumPy or Arrow C code.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"a": np.arange(1_000_000), "b": np.arange(1_000_000)})

# Slow – Python-level call per row
df["c"] = df.apply(lambda row: row["a"] + row["b"], axis=1)

# Fast – vectorized
df["c"] = df["a"] + df["b"]
```

The interview probe is whether the candidate reaches for vectorization first and only falls back to `apply` when the operation cannot be vectorized. A candidate who reaches for `apply` first is signalling they have not run their code on real data.

When `apply` IS the right answer:

- Per-row logic that depends on multiple columns in a way that no NumPy or pandas vectorized form expresses cleanly.
- Calling an external API per row (in which case the bottleneck is the API, not the apply).
- Calling a Python library function that does not have a vectorized form.

The 2026 senior follow-up: "what alternatives do you reach for before `apply`?" The expected answer includes `np.vectorize` (faster than `apply` because it avoids the Series construction), `df.eval(...)` (uses `numexpr` for arithmetic), groupby-flavoured operations where possible (`df.groupby(...).transform(...)` is often faster than `df.apply(...)`), and as a fallback for the worst case, Cython or Numba.

How interviewers ask this: "Your colleague wrote this and it is slow — speed it up." The interviewer shows an apply-based snippet on a million-row frame. The 2026 expected answer is "vectorize this," and the senior follow-up is "what if vectorization is genuinely impossible — what do you do then." A candidate who says "use `apply` with `swifter` or `pandarallel`" without first vectorizing is missing the point. Those tools accelerate `apply` by a constant factor (typically 2-8x). Vectorization is a different complexity class.

What saying it correctly tells them: You have profiled real pandas code. You know when `apply` is the bug and when it is unavoidable.

5.5 Other pandas 2.x items worth knowing

For completeness:

- **pd.read_parquet with Arrow** — Parquet is the default columnar format for data engineering. pandas 2.x reads Parquet via **pyarrow** by default. Asked as "how do you store a DataFrame for downstream use" — Parquet is the 2026 answer, CSV is the 2020 answer.
- **pd.concat performance** — concatenating many frames is slow because each concat is $O(n+m)$. Using a list-of-frames and a single **pd.concat** at the end is the right pattern. Asked as "your loop is slow — why" with a snippet that concatenates inside the loop.
- **pd.merge with validate=** — pandas 0.21+ exposes a **validate=** keyword that asserts the join cardinality (**"one_to_one"** , **"one_to_many"** , **"many_to_one"** , **"many_to_many"**). The feature has been around since 2017 but most candidates still don't reach for it. Using it correctly catches the silent-explosion bug where a many-to-many merge produces 10x rows. Senior-tier signal.
- **groupby with as_index=False** — small but interview-relevant. Returns groupby results without setting the group keys as the index. Asked when the result of a groupby needs to feed into a subsequent operation that expects flat columns.
- **pd.options.future.infer_string** — a forward-looking option that uses pyarrow strings by default. This will become the default in pandas 3.0. Senior candidates flag this as a "I read the migration guide" signal.

The composite 2026 pandas question that touches all of these: "load this 5 GB CSV, clean the missing values, group by user, compute the rolling mean of one column over a 7-day window, and write the result back as Parquet." A senior candidate writes this with Arrow-backed loading, CoW-aware semantics, **.isna()** for missing-value handling, vectorized rolling, and Parquet output. A junior candidate writes the same logic with **apply** somewhere in the middle and a CSV write at the end.

Section 6 — What interviewers stopped asking in 2026

The other side of the "what changed" report: what has been quietly retired. If your prep time is finite, deprioritize the items below. They are no longer signal — answering them well does not move the rubric.

6.1 Python 2 vs Python 3 compatibility

Dead. Completely. Python 2 reached end of life in January 2020. By mid-2026 there is no living interviewer at an Indian product co or GCC who is asking "what are the differences between Python 2 and 3." The question is gone. The classic answer points — **print** as a statement vs function, integer division, unicode by default, **range** vs **xrange** , **dict.iteritems** removal — are no longer asked because the assumption is that the candidate has only ever written Python 3.

The single edge case where you still might encounter Python 2: legacy Service MNC client work on Red Hat 7 systems that have not been migrated. If your interview is for a specific BFSI maintenance role on a legacy stack, the recruiter will tell you in advance. Otherwise, ignore. Allocate the prep time elsewhere.

The corollary: do not bring up Python 2 in your interview answers. Mentioning "this changed from Python 2" when discussing any Python feature is a tell that you learned Python before 2018 and you have not refreshed your mental model. If you are talking about how a current feature works, talk about how it works in 3.12. Don't anchor on the historical contrast.

6.2 Basic list comprehension rote

Dying fast at mid-level and above. List comprehensions are still asked at fresher rounds — they are baseline literacy — but the depth has changed. The 2024 question "rewrite this for loop as a list comprehension" is no longer signal at mid-level. Every candidate can do it.

What has replaced it:

- "When is a list comprehension the wrong choice?" — expected answer involves generators (memory), readability for complex transforms, or composition with the Walrus operator inside the comprehension.
- "Rewrite this list comprehension as a generator expression — why might you?"
- "What changed about comprehension scoping between Python 2 and Python 3?" — the loop variable used to leak in Python 2; in Python 3 it is scoped. Note that this still gets asked occasionally as a way to probe whether the candidate has internalized comprehension scoping; the right answer is "in Python 3 the loop variable is local to the comprehension, which is a deliberate fix."

The fresher round still asks "rewrite this for loop as a list comprehension." The mid-level round asks the questions above. The senior round asks comprehension-scoping traps as Guess-the-Output puzzles (see File 05).

Prep allocation rule: if you have 2 hours and you are mid-level or above, spend zero of it on basic comprehension rewriting. Spend it on the comprehension-scoping traps and the generator-comprehension tradeoff.

6.3 The pre-async threading vs multiprocessing rote answer

Two years ago the canonical concurrency question was "explain the difference between threading and multiprocessing in Python." The textbook answer was "threading is for I/O-bound, multiprocessing for CPU-bound, the GIL is why this matters." That answer is no longer enough.

In 2026 the question has expanded to a three-way comparison: threading, multiprocessing, and asyncio. The expected answer routes the question by workload shape: many concurrent I/O operations go to asyncio if the library supports it and to threading if not; CPU-bound parallel work goes to multiprocessing; small concurrent I/O without a structural concurrency story goes to threading.

A candidate who recites the two-way threading-vs-multiprocessing answer in 2026 is signalling that they learned Python concurrency before 2020 and stopped. The interviewer will follow up with "what about asyncio?" and the candidate's response to that follow-up determines the bucket.

The corollary: the GIL question itself has changed shape (see the Executive Summary). The 2024 GIL answer was definitional. The 2026 GIL answer includes the comparative threading-vs-multiprocessing-vs-asyncio routing AND the forward-looking PEP 703 / PEP 684 context. Recite only the definitional part and you are dating yourself.

6.4 `__metaclass__ = type` and Python 2 metaclass syntax

Gone. Completely. Even when metaclasses come up in modern Python interviews — and they do, occasionally, at the senior end — the question is "give me a use case for metaclasses" or "explain `type()` as both a function and a metaclass." Nobody asks about the Python 2 `__metaclass__ = type` class-body declaration any more. The Python 3 `class Foo(metaclass=Bar):` syntax is the assumed default.

Metaclass questions in 2026 are about understanding, not about syntax: "give me a real-world use case for a metaclass," "what does Django's `ModelBase` metaclass actually do," "how would you enforce a constraint at class-creation time." Answering these requires understanding what metaclasses are FOR. Answering the Python 2 syntax question requires knowing trivia from 2008.

6.5 Stale or low-signal questions that were retired in the last twelve months

A small list of patterns that have visibly faded:

- **"Explain the difference between `__init__` and `__new__`"** — still asked at fresher rounds, gone at mid+. The mid-level version is "give me a use case for `__new__`" — singleton, immutable type subclass.
- **"What are `*args` and `kwargs`"** — fresher round only.
- **"Is Python pass-by-value or pass-by-reference"** — was asked everywhere two years ago; now mostly asked at fresher rounds. The semantic answer is "pass-by-object-reference" or "pass-by-assignment," and the expected explanation involves mutability and rebinding, not the procedural distinction.
- **"Explain what a Python module is"** — gone except at the lowest screening rounds.
- **"What is PIP"** — gone. Replaced by `uv` and `pip-tools` related questions at the senior end, but the original "what is PIP" is screening-only.
- **"Explain `is` vs `==`"** — gone as a standalone question; it has migrated into the Guess-the-Output traps (small-integer cache, string interning), where it is a much sharper signal.
- **Rote GIL recitation** — gone; expect synthesis questions instead. See Section 6.3 above for the shape the GIL question takes in 2026.

The pattern across all of these: questions where the right answer is recitation of a textbook definition have been retired. Questions where the right answer requires applied reasoning have replaced them.

6.6 What this means for your prep

The interview market has shifted from "do you know this exists" questions toward "can you reason about which option to use and why" questions. The death of trivia questions has been one of the most encouraging shifts of the last two years for serious candidates. The flip side: candidates who studied off year-old YouTube playlists or off 2022-era PDFs are walking into rounds prepped for a question diet that has moved on.

If your competitor PDF prep is heavy on Python 2 vs 3 differences, on basic comprehension rewriting, on definitional GIL explanations, or on `__init__` vs `__new__` — those pages are not where the 2026 interview rounds live. The pages where the 2026 rounds live are decorators, generators, async with `TaskGroup`, typing fluency, pandas 2.x, and the Guess-the-Output traps in File 05.

Allocate accordingly.

Section 7 — Sourcing methodology

This section establishes what this report is and what it is not. It is intentionally short. Pages of methodology are not the value; the intel itself is the value.

7.1 What this report is

This is a digest of patterns we have heard about from candidates, hiring managers, and senior engineers running rounds at Indian companies between June 2025 and May 2026. The signal sources are listed below in descending order of weight.

Source 1 — Candidate-reported interview content. The single largest input. Candidates who buy our prep tell us what they were actually asked after their rounds, often within a few days. We do not get individual interview leaks; we get pattern data — which question shapes are recurring across companies, which are dying off, which are new. The pattern data is what feeds the "X% of mid-level Razorpay rounds asked about Y" claims in this report.

This source has two structural biases worth being honest about. First, we hear more from candidates who pass than from candidates who fail; failure rate underreporting is real. We compensate by aggregating across many candidates per company. Second, we hear more about distinctive questions than about generic ones — the candidate who got asked about `TaskGroup` is more likely to tell us than the candidate who got asked about list comprehensions. We compensate by treating the absence of "I got asked about X" reports as informative, not just noise.

Source 2 — Hiring manager and senior engineer conversations. People on our network who run rounds at the companies in this report. They tell us what they have added, dropped, or reweighted in their question banks in the last quarter. This is qualitative input, not quantitative. It catches shifts before the candidate-side data does — when a senior engineer at a product co tells us "we added Pydantic v2

questions to our screening rubric three months ago," that is a forward-looking signal that the question will start appearing in candidate reports six to twelve weeks later.

Source 3 — Official PEP and Python release notes. When a feature like `TaskGroup` (PEP 654) or `Self` (PEP 673) graduates from "proposed" to "shipped in CPython," we can be reasonably confident it will start appearing in interview questions within twelve to eighteen months. That correlation has held up well across the last five years. We track PEP and changelog signal as a leading indicator of where the question bank will move next.

Source 4 — Framework release notes. Pydantic v2 release notes, FastAPI release notes, Django release notes, pandas release notes. Each major version of a framework that is broadly adopted produces a wave of "have you migrated" questions in interview rounds for the following twelve to eighteen months. We track these the same way we track PEPs.

Source 5 — Ad-research signal from competitor scrapes. Where competitor prep products are advertising specific topics ("Master Python 3.13 async!"), that tells us what they think will sell, which is itself a signal about where the market believes the prep gap is. We weight this lowest because the competitor's read of the market is sometimes wrong, and because their advertising selection is shaped by their inventory rather than by interview reality.

7.2 What this report is NOT

A few things this report deliberately is not, in the interest of calibration.

It is not a survey with a p-value. When we say "appearing in roughly half of rounds we hear about at Razorpay," we mean we have heard this from enough candidates that we are reasonably confident the pattern is real, not that we ran a statistically rigorous survey. The "roughly" carries real weight.

It is not a question leak. We do not publish interview questions verbatim. We aggregate the *shape* of what is being asked. Specific phrasings in this report are illustrative reconstructions, not transcripts.

It is not exhaustive. There are patterns we don't see because we don't have visibility into every company. Smaller GCCs, very early-stage startups, and most consulting-bench rounds are underrepresented in our signal. We try to flag where coverage is thin.

It is not future-proof beyond six months. The Python ecosystem moves fast enough that this report has a half-life of six months. We refresh it twice a year for exactly this reason. The H1 2027 edition will look meaningfully different.

7.3 What counts as a "real" signal vs noise

A working definition we use internally:

- **Strong signal:** heard from three or more independent candidates at the same company within the last 90 days, OR confirmed by a hiring manager at that company in the last 90 days, OR backed by a published PEP/release note that shipped 12+ months ago and has had time to propagate.

- **Moderate signal:** heard from one or two candidates within the last 90 days, OR heard from multiple candidates within the last 180 days, OR backed by a PEP/release note that shipped 6-12 months ago.
- **Weak signal:** single-candidate report older than 90 days, OR forward-looking projection based on a PEP/release that has not yet shipped, OR competitor-advertising signal alone.

We promote strong signal to the body of this report. Moderate signal we mention with hedging ("climbing," "showing up in some rounds"). Weak signal we either omit or flag explicitly as forward-looking.

This calibration is why the report does not claim "everyone is asking about the 3.13 JIT" — the signal volume is moderate and the JIT itself is experimental. We mention it; we don't promote it.

7.4 The refresh cadence

We refresh this file every six months. Specifically:

- **Late May** — H1 refresh. Captures the January-to-May interview cycle (which is heavy in India — fiscal year end at clients drives mid-cycle hiring at Service MNCs, and Q1 internal review cycles at product cos surface a wave of attrition that gets backfilled in Q2).
- **Late November** — H2 refresh. Captures the July-to-November interview cycle (campus and lateral hiring at product cos and GCCs, who run their heaviest hiring waves in August-October).

First edition: May 2026 (this report). Next refresh scheduled for November 2026, with a follow-up in May 2027. The Python language pack is new in May 2026; earlier language packs in our catalogue carried their own intel files on the same 6-month cadence, and the Python edition picks up that cadence from the start.

What gets re-evaluated at each refresh:

1. Every claim with a "now baseline" or "appearing in X% of rounds" framing is re-validated against the most recent 90 days of candidate reports. Claims that no longer hold up get downgraded or removed.
2. Patterns that were "climbing" in the previous edition are re-evaluated. If they have stabilized, they move into the "now baseline" body. If they have not climbed further, they get downgraded.
3. The "stopped asking" section gets refreshed — anything that has come back into rotation gets added back to the body.
4. Newly-shipped PEPs and framework versions get evaluated for likely interview impact and added to the forward-looking sections.
5. The executive-summary top-six list gets re-ranked based on signal volume in the last 90 days.

7.5 The pack-level promise

The whole prep pack carries the same "refreshed every 6 months" promise that this file makes specifically. The other intel files (01 Top 50 and 02 Top 200) are refreshed at the same cadence —

questions that have moved out of the top-50 or top-200 most-asked list get rotated out, new patterns get rotated in. The library files (04 Core, 06 Data Track, 07 Backend Track) get smaller incremental refreshes — new chapters added when a major language feature ships, old chapters trimmed when patterns retire.

Buyers who downloaded the pack in November 2025 should re-download from their Drive link in late May 2026 to pick up this edition. The Drive folder URL does not change between editions; the individual PDFs inside do. Future editions will follow the same pattern.

That is the full sourcing model. If anything in the body of this report contradicts what you are seeing in your own interview rounds, log the gap and we will catch it in the next refresh — that is exactly the candidate-report signal that feeds Source 1.

End of File 03 — What Changed in 2026. Continue to File 04 (Core Python Library, 600 Qs) for the systematic deep prep, or to File 05 (Guess the Output, 50 Pythonic Traps) for the signature differentiator.